DOCUMENTACIÓN TÉCNICA OFICIAL

Planificador de Compensatorios

Guía paso a paso: De Excel local a Aplicación Web Segura en Producción (Vercel + Supabase)

📖 Bitácora Paso a Paso: Cómo se Creó y Desplegó esta Aplicación

Esta guía documenta de principio a fin todo el proceso técnico, arquitectónico y de seguridad que seguimos para transformar una hoja de cálculo local en una **aplicación web moderna, segura, en tiempo real y 100% gratuita** desplegada en internet.

📑 Tabla de Contenidos

1. El Problema Inicial y Diagnóstico
2. Arquitectura Elegida (Seguridad Primero)
3. Construcción del Frontend y Backend
4. Configuración de la Base de Datos en Supabase
5. Control de Versiones y Subida a GitHub
6. Despliegue Continuo en Vercel
7. Auditoría de Seguridad y Blindaje
8. Cómo Usar y Mantener la Aplicación

---

1. El Problema Inicial y Diagnóstico

El Punto de Partida

Teníamos dos archivos en local: * `Control_Horas_Compensacion - Anderson CT.xlsx` : Archivo Excel con los registros de compensación y la meta de 40 horas. * `planificador_de_horas_de_compensaci_n_enero_2027 (1).html` : Un prototipo visual en HTML con Tailwind CSS.

El Obstáculo

El aplicativo guardaba los cambios únicamente en el `localStorage` del navegador web de esa máquina específica. * Si abrías el archivo en otro computador o en el celular, aparecía vacío o desactualizado. * Las marcas de "Cumplido" hechas en la oficina no se reflejaban en la casa.

---

2. Arquitectura Elegida (Seguridad Primero)

El objetivo era tener una versión **online, multi-dispositivo y gratis**, pero con un requisito no negociable: **cero credenciales expuestas en el navegador**.

Esquema de la Arquitectura

```
[ NAVEGADOR WEB (Cualquier PC o Celular) ]
                    |
                    ▼ (Llamadas limpias sin contraseñas ni URLs)
              /api/sessions
                    |
                    ▼
[ BACKEND SERVERLESS (Vercel Node.js Function) ]
   🔒 Lee variables secretas de entorno (process.env):
      • SUPABASE_URL
      • SUPABASE_KEY
      • APP_PIN (Opcional)
                    |
                    ▼ (Conexión cifrada directa)
[ BASE DE DATOS EN LA NUBE (Supabase PostgreSQL) ]
```

Tecnologías Seleccionadas (100% Gratuitas)

- **Frontend**: HTML5, Tailwind CSS (vía CDN), FontAwesome 6 y Vanilla JavaScript.
- **Backend**: Serverless Function en Node.js sobre **Vercel**.
- **Base de Datos**: PostgreSQL gestionado en la nube con **Supabase** (Plan Free).
- **Control de Código**: **GitHub**.

---

3. Construcción del Frontend y Backend

A. El Frontend (`index.html`)

Diseñamos una interfaz ejecutiva con paleta sobria inspirada en la Legión de Reconocimiento / Alas de la Libertad (verde bosque `#1E382B`, cuero cálido `#6E4C33` y piedra suave `#EFECE5`): * **Cálculo Estricto en Enteros**: Cero decimales para cumplir normas laborales (40h totales: 20h Teletrabajo y 20h Presencial). * **Cobertura de Enero 2027**: Tarjetas para los 5 días laborales (4 al 8 de enero de 2027, 8h cada uno). * **Atajos Rápidos a 1 Clic**: * Sábado Presencial (6h): Sábados de 7:00 am a 1:00 pm. * Antes de Jornada (1h): 6:30 am a 7:30 am (Teletrabajo). * Final de Jornada (1h): 4:20 pm a 5:20 pm (Teletrabajo). * **Motor Inteligente de Auto-planificación**: Calcula el déficit exacto y programa los días faltantes sin solapamientos. * **Conexión Limpia**: Llama únicamente a `/api/sessions`, sin saber qué base de datos existe detrás.

B. El Backend (`api/sessions.js`)

Creamos una función serverless que actúa como intermediaria segura: * Responde a peticiones `GET` (lectura) y `POST` (guardado). * Convierte automáticamente entre la nomenclatura de JavaScript (`horaInicio`, `horaFin`) y las columnas de PostgreSQL (`hora_inicio`, `hora_fin`). * Conecta con la API PostgREST de Supabase usando `process.env.SUPABASE_URL` y `process.env.SUPABASE_KEY`.

---

4. Configuración de la Base de Datos en Supabase

1. Se creó el proyecto `compensatorios` en supabase.com.
2. En el **SQL Editor**, se ejecutó el script `supabase_schema.sql` : ```sql CREATE TABLE IF NOT EXISTS compensatorios_sessions (id TEXT PRIMARY KEY, fecha TEXT NOT NULL, dia TEXT NOT NULL, hora_inicio TEXT

NOT NULL, hora_fin TEXT NOT NULL, horas INTEGER NOT NULL DEFAULT 1, modalidad TEXT NOT NULL, estado TEXT NOT NULL, observaciones TEXT DEFAULT '', updated_at TIMESTAMPTZ DEFAULT now());

ALTER TABLE compensatorios_sessions ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Permitir acceso completo a la API de compensatorios" ON compensatorios_sessions FOR ALL USING (true) WITH CHECK (true); `` 3. Se insertaron los 5 registros iniciales del Excel. 4. Se obtuvieron las 2 claves necesarias: * ** SUPABASE_URL **: https://vpiygubqwfsdtvojomkk.supabase.co * ** SUPABASE_KEY **: La Publishable Key (anon public`).

---

5. Control de Versiones y Subida a GitHub

1. Se inicializó el repositorio local con `git init -b main`.
2. Se vinculó la llave pública de la máquina (`id_ed25519.pub`) en GitHub Settings.
3. Se enlazó el repositorio remoto: `bash git remote add origin https://github.com/19ufo/compensatorios.git git push -u origin main`
4. Se subieron todos los archivos del proyecto a GitHub.

---

6. Despliegue Continuo en Vercel

1. En vercel.com, se importó el repositorio `19ufo/compensatorios`.
2. En la sección **Environment Variables**, se configuraron las claves secretas:
3. `SUPABASE_URL` = `https://vpiygubqwfsdtvojomkk.supabase.co`
4. `SUPABASE_KEY` = *(Clave anon de Supabase)*
5. **Ajuste clave de enrutamiento**: Inicialmente Vercel mostraba `Modo Local` porque `vercel.json` contenía una directiva obsoleta (`routes`). Se simplificó `vercel.json` a: `json { "cleanUrls": true }` Esto activó de inmediato el enrutamiento automático de Serverless Functions hacia `/api/sessions`.
6. El aplicativo quedó publicado en producción en: 👉 `https://compensatorios.vercel.app`

---

7. Auditoría de Seguridad y Blindaje

Realizamos una auditoría técnica completa y aplicamos medidas de protección contra los principales riesgos de seguridad web (OWASP Top 10):

1. **Protección contra Cross-Site Scripting (XSS)**:
2. Se crearon las funciones `escapeHtml()` y `sanitizeId()` en el frontend.
3. Cualquier texto en observaciones o identificadores se codifica antes de insertarse en el DOM (`&` , `<` , `>` , `"`).
4. **Mitigación de Inyección de Fórmulas en CSV**:
5. Al exportar a CSV, si una observación inicia con `=` , `+` , `-` o `@` , se antepone una comilla simple (`'`) para que Excel lo trate como texto inofensivo.
6. **Restricción de Origen (CORS)**:
7. El backend solo acepta peticiones provenientes de dominios autorizados de Vercel y entornos locales de desarrollo.

8. **Protección contra DoS y Carga Masiva**:
9. Límite estricto de máximo 100 sesiones por petición y 500 caracteres por observación.
10. Validación de tipos con listas blancas (`Teletrabajo` / `Presencial` , `Cumplido` / `Planificado` , horas entre 1 y 24).
11. **Cabeceras de Seguridad HTTP (en `vercel.json`)**:
12. `X-Frame-Options: DENY` : Evita ataques de Clickjacking (que otra página incruste tu app en un iframe).
13. `X-Content-Type-Options: nosniff` : Evita que el navegador interprete archivos con tipos MIME incorrectos.
14. `Referrer-Policy: strict-origin-when-cross-origin` : Protege la privacidad en las cabeceras de navegación.

---

8. Cómo Usar y Mantener la Aplicación

- **Desde el Celular o Computador**: Abre `https://compensatorios.vercel.app` en cualquier navegador. Verás el indicador 🟢 `Backend Seguro` .
- **Marcar Horas**: Haz clic en el botón de estado de cualquier fila para alternar entre **Planificado** y **Cumplido**. Los cambios se envían automáticamente al backend y se guardan en Supabase.
- **Ver los Datos en Crudo**: Puedes entrar a tu panel de **Supabase** → **Table Editor** → `compensatorios_sessions` para ver o editar tus registros como si fuera un Excel en la nube.
- **Respaldos**: La aplicación incluye botones para descargar copias en formato **CSV** (para Excel) y **JSON** (para respaldo completo).
- **Despliegues Futuros**: Cualquier cambio que hagas en el código local y subas con `git push origin main` se actualizará automáticamente en Vercel en menos de 15 segundos.

Fin de la bitácora técnica — Desarrollado para Anderson CT (Enero 2027).
